TEAM 7 INFRASTRUCTURE

Job Queue & Resource Allocation Design Document

Version 2.0

Technical Design Specification | Slurm Workload Manager

Document attribute	Description
Project	Team 7 - Infrastructure Queue Management
Scheduler	Slurm Workload Manager
Development environment	macOS, Docker Desktop, Slurm Docker Cluster
Development compute nodes	c1, c2
Test users	user1, user2
Target production infrastructure	Single 8× NVIDIA H100 GPU node
Workload categories	Debugging, experimentation, model training
Design status	Proposed architecture and validation plan

Docker validation to a single 8-H100 GPU node

Contents

Complete design specification - 17 sections

1. Executive Summary . . . 3

2. System Architecture . . . 4

3. Queue and Partition Design . . . 6

4. Slurm Configuration Design . . . 7

5. User Management and Fair-Share Policy . . . 10

6. Job Submission Templates . . . 12

7. Job Lifecycle and Scheduling States . . . 15

8. Multi-User Concurrency and Queue Verification . . . 16

9. Queue Monitoring and Operational Commands . . . 19

10. Failure Handling and Recovery . . . 21

11. Logging, Auditability, and Retention . . . 23

12. Security and Isolation Controls . . . 24

13. Deployment and Configuration Change Management . . . 24

14. Docker-to-H100 Migration Plan . . . 25

15. Risks, Limitations, and Open Decisions . . . 26

16. Project Deliverables and Acceptance Criteria . . . 27

17. Conclusion . . . 28

1. Executive Summary

This document defines the architecture, configuration, security controls, scheduling policies, operational procedures, and validation strategy for Team 7's Slurm-based infrastructure queue management system.

The system is designed to support multiple independent users submitting computational workloads while ensuring predictable resource allocation, fair scheduling, workload isolation, and operational visibility.

The implementation follows two stages:

Stage 1 - Docker-based functional validation

A containerized Slurm cluster running on macOS is used to verify scheduler behavior independently of physical GPU infrastructure. Two independent users (user1 and user2) submit jobs to the c1 and c2 compute nodes, allowing the team to validate concurrency, resource contention, pending queues, accounting, cancellation, and failure handling.

Stage 2 - Production GPU deployment

The validated scheduling architecture will be adapted for an infrastructure node containing eight NVIDIA H100 GPUs. Slurm will allocate CPU cores, memory, and GPU resources to workloads through explicit resource requests and GPU Generic Resources (GRES).

The production design introduces persistent accounting, fair-share scheduling, resource quotas, security controls, monitoring, and recovery procedures.

1.1 Design objectives

1. Provide reproducible job submission through sbatch.
2. Support multiple users without exposing privileged job-management operations across accounts.
3. Separate short debugging workloads from long-running AI model training.
4. Schedule workloads according to CPU, RAM, GPU, and runtime requirements.
5. Implement fair-share prioritization to discourage resource monopolization.
6. Maintain auditable job histories and failure diagnostics.
7. Support progression from local Docker simulation to production GPU infrastructure without redesigning the scheduling model.

1.2 Scope and assumptions

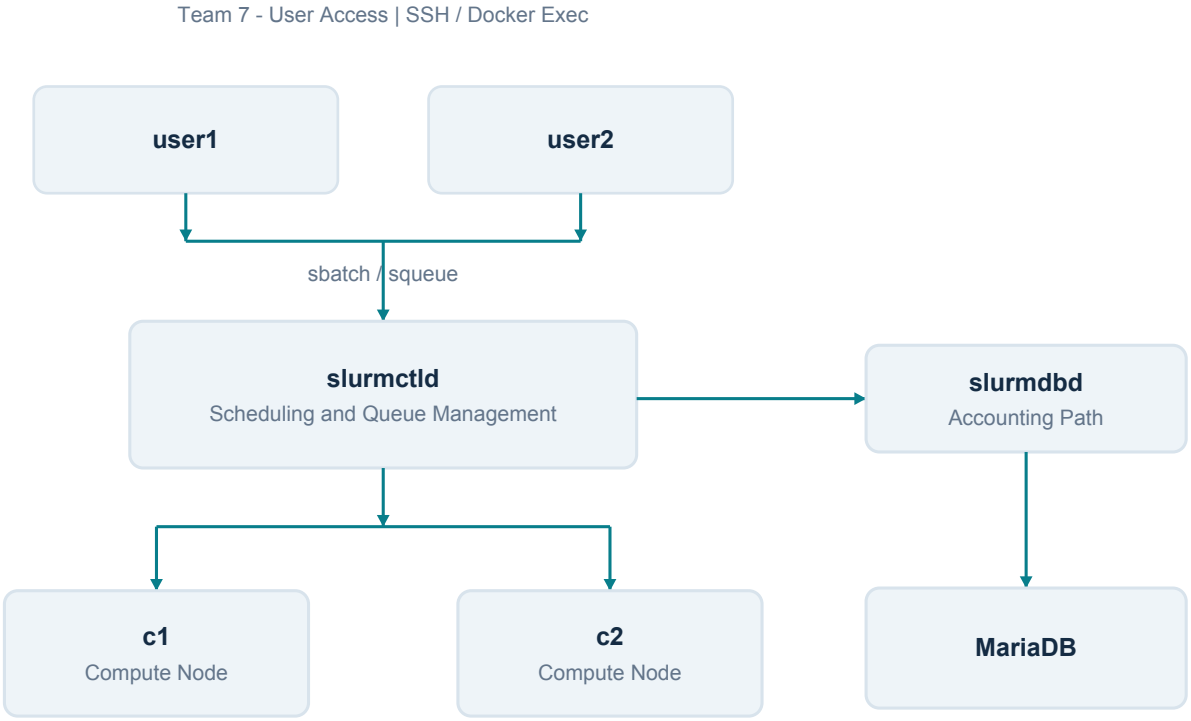
The design covers Slurm scheduling, user accounts, partitions, resource allocation, monitoring, accounting, logging, and common failure conditions.

The following assumptions apply:

- The Docker environment has a controller (slurmctld), accounting service (slurmdbd), database (mariadb), and two compute nodes (c1 and c2).
- The local environment primarily validates CPU and memory resource scheduling; it does not establish that H100 GPU scheduling works.
- The production system initially contains one compute node with eight NVIDIA H100 GPUs.
- Production CPU count, memory capacity, exact H100 variant, GPU topology, network fabric, and storage specifications must be discovered before the final resource configuration is applied.
- All users have appropriate Unix accounts and consistent identity mappings across cluster components.
- Persistent storage and backup locations will be provisioned separately from ephemeral container storage.

2. System Architecture

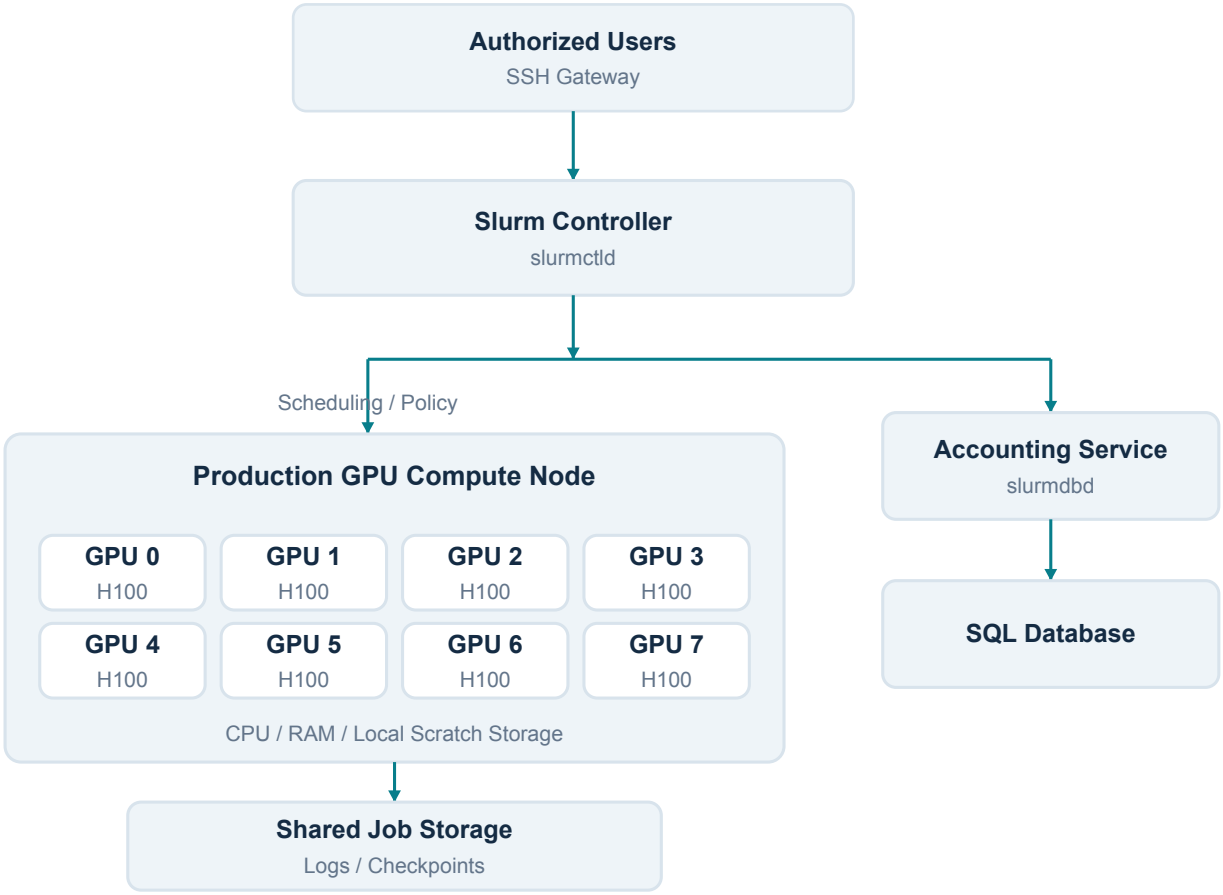
2.1 Development architecture



The controller receives job submissions, evaluates partition and resource constraints, schedules eligible workloads, and instructs compute nodes to launch jobs.

The accounting service records submitted workloads, resource consumption, job states, and historical usage information.

2.2 Production architecture



In a single-node pilot, the controller may be colocated with the GPU server if required, but a dedicated controller and durable accounting database are preferable for production reliability.

2.3 Component responsibilities

Component	Responsibility
slurmctld	Job admission, scheduling, priority evaluation, and node management
slurmd	Compute-node daemon that communicates with the controller
slurmstepd	Execution and tracking of job steps
slurmdbd	Persistent job accounting and association management
MariaDB	Stores Slurm accounting information
Munge	Authentication between Slurm components
c1, c2	Docker development compute nodes
GPU compute node	Executes production AI and ML workloads
Shared storage	Provides datasets, logs, checkpoints, and job scripts

3. Queue and Partition Design

Slurm partitions represent logical job queues with configurable resource boundaries and scheduling policies.

Team 7 will implement two primary partitions: debug and train.

3.1 Partition definitions

Parameter	debug	train
Purpose	Diagnostics, smoke tests, troubleshooting	Full training and longer experiments
Maximum wall time	30 minutes	48 hours
Default wall time	10 minutes	4 hours
Maximum GPUs per job	1 (proposed policy)	8
CPU allocation	Small, controlled requests	Workload-dependent
Memory allocation	Small, controlled requests	Workload-dependent
Priority	Higher short-job partition factor	Standard partition factor
Preemption	Disabled initially	Disabled initially
User access	Authorized development users	Authorized training users
Primary goal	Quick turnaround	Efficient training throughput

Important scheduling distinction: Partition priority does not guarantee immediate execution. A pending debug job cannot automatically displace a running training job unless an explicit preemption policy is enabled.

For the initial production deployment, disabling preemption avoids interrupting expensive training jobs. Debug responsiveness will depend on resource availability, backfill scheduling, and optionally reserved resources.

3.2 Resource allocation policies

All users must declare their expected resource consumption. The scheduler should not allocate more resources than requested, and jobs should be constrained to their allocated resources using the appropriate task and cgroup plugins.

For example, a production training job requesting four GPUs should not be able to consume or access GPUs allocated to another job.

Resource requirements are evaluated using:

- CPU cores: --cpus-per-task, --ntasks
- Memory: --mem or --mem-per-cpu
- GPUs: --gres=gpu:N or an appropriate Slurm GPU request option
- Runtime: --time
- Partition: --partition
- Account: --account, if accounting associations are enforced

3.3 Shared-node partition behavior

On a single 8-H100 server, both debug and train partitions may reference the same physical node.

This means the partitions provide different policies but do not create physically independent resources.

A training job that allocates all eight GPUs can prevent a new debugging job from starting, even if the debug partition has a higher priority factor.

For strict debug availability, Team 7 must either reserve appropriate resources, use a deliberate preemption strategy, or introduce separate debug hardware.

The recommended initial model is a shared node with fair-share scheduling and backfill, accepting that debug jobs may occasionally wait.

4. Slurm Configuration Design

4.1 Core slurm.conf configuration

The following configuration provides a baseline for the proposed scheduler. It is intended as a reference snippet to merge into the existing Docker cluster configuration, not as a replacement for its complete working slurm.conf.

```
# ========================================
# Team 7 - Slurm Core Configuration
# ========================================

ClusterName=team7-cluster

# Scheduling
SelectType=select/cons_tres
SelectTypeParameters=CR_Core_Memory

SchedulerType=sched/backfill

# Multi-factor priority scheduling
PriorityType=priority/multifactor
PriorityDecayHalfLife=7-00:00:00
PriorityMaxAge=7-00:00:00

# Priority contributions
PriorityWeightAge=1000
PriorityWeightFairshare=10000
PriorityWeightJobSize=0
PriorityWeightPartition=1000

# Logging and accounting
AccountingStorageType=accounting_storage/slurmdbd
AccountingStorageEnforce=associations,limits

JobAcctGatherType=jobacct_gather/cgroup
JobAcctGatherFrequency=task=30

# Resource isolation
ProctrackType=proctrack/cgroup
TaskPlugin=task/cgroup

# Job handling
ReturnToService=1
KillWait=30
MinJobAge=300
```

Configuration notes:

- SelectType=select/cons_tres enables resource selection using trackable resources such as CPUs, memory, and GPUs.
- SelectTypeParameters=CR_Core_Memory allocates consumable CPU cores and memory.
- SchedulerType=sched/backfill enables eligible smaller jobs to run when doing so will not delay higher-priority reservations.
- PriorityType=priority/multifactor evaluates several factors instead of scheduling exclusively by submission order.
- PriorityWeightFairshare=10000 gives historical usage a significant influence on queue priority.
- AccountingStorageEnforce=associations,limits requires valid accounting associations and enforces configured association/QOS limits.
- Cgroup task tracking supports job isolation and resource usage accounting when configured correctly.

The existing cluster must also retain valid SlurmctldHost, AuthType, SlurmdUser, node definitions, SlurmctldLogFile, and other installation-specific settings.

Production prerequisites: Functional cgroup tracking requires a matching cgroup.conf, compatible system cgroups, and suitable Slurm container privileges. Configuration directives alone do not establish resource isolation.

4.2 Docker partition configuration

Assuming c1 and c2 are already defined as valid compute nodes:

```
# ========================================
# Docker Development Partitions
# ========================================

PartitionName=debug \
    Nodes=c1,c2 \
    Default=YES \
    MaxTime=00:30:00 \
    DefaultTime=00:10:00 \
    PriorityJobFactor=200 \
    State=UP

PartitionName=train \
    Nodes=c1,c2 \
    Default=NO \
    MaxTime=2-00:00:00 \
    DefaultTime=04:00:00 \
    PriorityJobFactor=100 \
    State=UP
```

The higher PriorityJobFactor gives the debug partition a larger partition-priority contribution when the multifactor scheduler uses partition weighting.

These partitions share the same two physical compute nodes. Their resources are therefore shared, not duplicated.

4.3 Production GPU partition configuration

For the 8-H100 production server, the partition definitions can use the same node under different workload policies.

```
# ========================================
# Production GPU Partitions
# ========================================

PartitionName=debug \
    Nodes=gpu01 \
    Default=YES \
    MaxTime=00:30:00 \
    DefaultTime=00:10:00 \
    PriorityJobFactor=200 \
    State=UP

PartitionName=train \
    Nodes=gpu01 \
    Default=NO \
    MaxTime=2-00:00:00 \
    DefaultTime=04:00:00 \
    PriorityJobFactor=100 \
    State=UP
```

The production gpu01 node must be declared separately using its discovered CPU topology, memory capacity, and GRES specification.

Illustrative syntax:

```
# Replace CPU and memory values with verified hardware data.
# Do not apply these illustrative values unchanged.

NodeName=gpu01 \
    CPUs=128 \
    RealMemory=1000000 \
    Gres=gpu:h100:8 \
    State=UNKNOWN

GresTypes=gpu
```

The example assumes 128 logical CPUs and approximately 1 TB of allocatable memory for illustration only. The actual configuration must match the physical node and resource-reservation policy. The H100 model name must match the declared GRES type.

4.4 GPU resource declaration with gres.conf

Example production /etc/slurm/gres.conf:

```
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia0
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia1
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia2
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia3
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia4
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia5
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia6
NodeName=gpu01 Name=gpu Type=h100 File=/dev/nvidia7
```

Before enabling GPU scheduling, administrators should verify:

```
nvidia-smi -L
nvidia-smi topo -m
slurmd -C
```

GPU discovery and MIG configuration require additional consideration if MIG is enabled. The configuration above assumes eight full GPUs rather than MIG instances.

4.5 Example cgroup.conf

```
# Resource isolation baseline

ConstrainCores=yes
ConstrainRAMSpace=yes
ConstrainDevices=yes
```

This file must be installed alongside the Slurm configuration, with platform-appropriate cgroup settings.

On production Linux, the administrator should verify that CPU affinity, memory enforcement, and GPU device visibility work under simultaneous users. Docker-based validation may require additional configuration and may not reproduce production kernel isolation.

5. User Management and Fair-Share Policy

5.1 User identities

The initial test environment contains two independent users:

Identity	Role	Access
user1	Test researcher A	Submit, view, and manage own jobs
user2	Test researcher B	Submit, view, and manage own jobs
Slurm administrator	Infrastructure operator	Manage nodes, jobs, partitions, and accounting policies

Each user should have a unique Unix UID, with consistent UID/GID mappings wherever jobs are submitted or executed.

Munge authentication must be configured consistently across the controller and compute nodes.

5.2 Slurm accounting associations

The users should belong to one shared Slurm accounting account so that usage is tracked under a common allocation while user-level fair-share behavior can be tested.

Run administrative commands on the controller, adjusting the cluster name to match the deployed configuration:

```
# Create the shared accounting account
sacctmgr -i add account team7 \
  Description="Team 7 Workloads"

# Create user associations
sacctmgr -i add user user1 \
  Account=team7

sacctmgr -i add user user2 \
  Account=team7

# Inspect associations
sacctmgr show assoc \
  format=Cluster,Account,User,Share,MaxJobs
```

The administrators must also verify that the Slurm cluster is registered in SlurmDBD and that the users' default account associations are correct.

5.3 Fair-share design

The desired behavior is to give users comparable scheduling opportunities over time rather than allowing one user to dominate GPU capacity.

A proposed starting policy:

Parameter	user1	user2
Fair-share allocation	Equal	Equal
Maximum active training jobs	Policy-controlled	Policy-controlled
Debug time limit	30 minutes	30 minutes
Training time limit	48 hours	48 hours
Relative scheduling privileges	Equal	Equal

Equal fair-share assignments do not imply alternating execution or equal immediate access to GPUs. Slurm fair-share prioritization reflects historical resource consumption, and effective job priority is also influenced by partition factors, job age, and other configured factors.

Administrators can inspect fair-share information using:

```
sshare -l

sprio -l
```

5.4 Optional QOS enforcement

Slurm Quality of Service (QOS) policies can implement operational limits beyond the partition definitions.

For example, the proposed policy is:

- Each user may have at most two simultaneously running jobs under a debug QOS.
- Each user may have at most one simultaneously running job under a train QOS.
- The debug QOS permits at most one GPU per job.
- The train QOS permits up to eight GPUs per job.

An illustrative administrative setup:

```
sacctmgr -i add qos team7_debug \
  MaxWall=00:30:00 \
  MaxJobsPU=2 \
  MaxTRESPerJob=gres/gpu=1

sacctmgr -i add qos team7_train \
  MaxWall=2-00:00:00 \
  MaxJobsPU=1 \
  MaxTRESPerJob=gres/gpu=8
```

Authorized users must be associated with these QOS definitions, and appropriate partition/QOS admission controls must be configured so users cannot bypass intended policy by choosing a different QOS.

QOS limits are only effective when accounting and enforcement are operational. For the Docker phase, the team should first validate the existing accounting configuration before introducing stricter admission controls.

6. Job Submission Templates

6.1 Debug partition - CPU diagnostics

File: debug_job.sh

```
#!/bin/bash
#SBATCH --job-name=team7-debug
#SBATCH --partition=debug
#SBATCH --account=team7
#SBATCH --time=00:05:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=512M
#SBATCH --output=logs/debug-%j.out
#SBATCH --error=logs/debug-%j.err

set -euo pipefail

echo "===== Team 7 Debug Job ====="
echo "Job ID: ${SLURM_JOB_ID}"
echo "User: $(whoami)"
echo "Node: $(hostname)"
echo "Start: $(date)"

echo "Running diagnostics..."
srun hostname
srun python3 --version

sleep 30

echo "Diagnostics completed"
echo "Finish: $(date)"
```

Before submitting the script:

```
mkdir -p logs

sbatch debug_job.sh
```

The logs directory must exist and be writable from the compute node, and the required Python executable must be available in the job environment.

Expected output:

```
Submitted batch job 101
```

The ID is illustrative; actual job IDs are assigned by Slurm.

6.2 Train partition - CPU workload simulation

This script is suitable for Docker testing when GPU hardware is unavailable.

File: train_simulation.sh

```
#!/bin/bash
#SBATCH --job-name=team7-train-test
#SBATCH --partition=train
#SBATCH --account=team7
#SBATCH --time=00:10:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=512M
#SBATCH --output=logs/train-%j.out
#SBATCH --error=logs/train-%j.err

set -euo pipefail

echo "===== Team 7 Training Simulation ====="
echo "Job ID: ${SLURM_JOB_ID}"
echo "User: $(whoami)"
echo "Node: $(hostname)"
echo "Start: $(date)"

echo "Simulating long-running training..."

srun sleep 180

echo "Simulation finished"
echo "Finish: $(date)"
```

Submission:

```
mkdir -p logs

sbatch train_simulation.sh
```

This establishes whether long-running jobs are queued and assigned correctly. It does not measure model-training performance.

6.3 Train partition - production H100 GPU workload

File: train_gpu.sh

```
#!/bin/bash
#SBATCH --job-name=team7-h100-train
#SBATCH --partition=train
#SBATCH --account=team7
#SBATCH --time=12:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=128G
#SBATCH --gres=gpu:h100:4
#SBATCH --output=logs/gpu-%j.out
#SBATCH --error=logs/gpu-%j.err

set -euo pipefail

echo "===== H100 Training Job ====="
echo "Job ID: ${SLURM_JOB_ID}"
echo "Allocated GPUs: ${SLURM_JOB_GPUS:-unknown}"

nvidia-smi

# Replace with the real distributed training entry point.
# The application must be configured for the allocated
# four GPUs and compatible process-launch strategy.

srun python3 train.py

echo "Training completed"
```

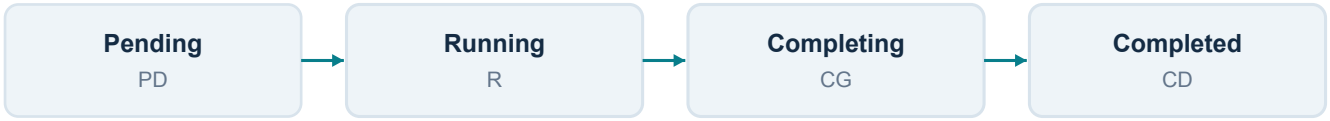
This example requests four full H100 GPUs, 16 CPUs, and 128 GB of RAM for a maximum of 12 hours.

It assumes a train.py program and environment are already installed. For PyTorch Distributed Data Parallel, the team must additionally configure the appropriate worker launch mechanism, rank assignments, and GPU device mapping.

GPU allocation is not equivalent to correct multi-GPU application execution; both must be tested.

7. Job Lifecycle and Scheduling States

A Slurm job progresses through a series of states from submission to completion.



Illustrative successful path; other outcomes are listed below.

Illustrative job lifecycle. Jobs can also be held, requeued, timed out, or terminated by infrastructure failures.

State	Slurm code	Description
Pending	PD	Waiting for resources, priority, or another scheduling condition
Running	R	Allocated workload is executing
Completing	CG	Job is finishing and resources are being released
Completed	CD	Job finished successfully
Failed	F	Job exited unsuccessfully
Cancelled	CA	Job was cancelled
Timeout	TO	Configured wall-time limit exceeded
Out of Memory	OOM	Job exceeded applicable memory constraints
Node Failure	NF	Compute node failed during execution

A pending job does not necessarily mean the cluster is malfunctioning. The scheduler may deliberately defer execution because of unavailable CPUs, memory, GPUs, QOS limits, or higher-priority reservations.

8. Multi-User Concurrency and Queue Verification

The primary functional test must demonstrate that two distinct user accounts can submit jobs independently and that Slurm correctly enforces compute-resource contention.

8.1 Prerequisites

Check the cluster:

```
sinfo -N -l
scontrol show nodes
scontrol show partition
```

Expected prerequisites:

- c1 and c2 appear as registered nodes.
- Both compute nodes are available for scheduling.
- Both partitions are visible.
- user1 and user2 have valid Unix identities.
- Slurm accounting permits both users to submit to team7.

8.2 Test A - Concurrent independent submissions

Submit a short debug job as user1 and another as user2, using separate authenticated sessions or suitable user switching.

```
# As user1
sbatch debug_job.sh
```

```
# As user2
sbatch debug_job.sh
```

Monitor all jobs:

```
squeue -u user1,user2
```

Expected behavior: Both users' jobs are accepted and may run simultaneously when sufficient resources exist.

8.3 Test B - Force a Pending state

To make contention reproducible, the test should target the same compute node and allocate it exclusively.

File: exclusive_test.sh

```
#!/bin/bash
#SBATCH --job-name=queue-contention
#SBATCH --partition=debug
#SBATCH --nodelist=c1
#SBATCH --exclusive
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --time=00:05:00
#SBATCH --output=slurm-%j.out

echo "Running on $(hostname)"
sleep 180
```

First, submit as user1:

```
sbatch exclusive_test.sh
```

Once the first job is running, submit the same script as user2:

```
sbatch exclusive_test.sh
```

Then monitor:

```
squeue -o "%.10i %.12u %.12P %.10T %.24R"
```

Expected result, assuming no conflicting limits or other workloads:

```
JOBID       USER         PARTITION    STATE      NODELIST(REASON)
101         user1        debug        RUNNING    c1
102         user2        debug        PENDING    (Resources)
```

This output is illustrative.

The second job should remain pending while c1 is exclusively allocated to the first job. When resources are released, it becomes eligible to run, subject to scheduling policy.

Critical test condition: The test only establishes resource contention when the first job has actually entered RUNNING. Submitting two jobs without checking the first allocation could produce different results.

8.4 Test C - Job ownership isolation

While user1 owns an active job, attempt cancellation as user2:

```
scancel <user1_job_id>
```

Expected behavior: A normal, unprivileged user2 account cannot cancel user1's job.

Repeat with the legitimate owner:

```
scancel <own_job_id>
```

Expected behavior: The owner can cancel their job.

This demonstrates control-plane authorization. Separate tests are required to establish filesystem permissions, memory isolation, and GPU device isolation.

8.5 Acceptance test matrix

Test ID	Scenario	Acceptance condition
T01	Node registration	Both c1 and c2 visible in sinfo
T02	Partition registration	debug and train available
T03	User 1 submission	Job accepted
T04	User 2 submission	Job accepted
T05	Concurrent execution	Both jobs run when resources permit
T06	Resource contention	Second exclusive job enters PD
T07	Resource release	Pending job becomes eligible to run
T08	Account isolation	Unauthorized cross-user cancellation rejected
T09	Runtime enforcement	Job exceeding partition limit terminates
T10	Accounting	Completed jobs visible in sacct
T11	Fair-share	Usage and priority factors observable
T12	Failure recovery	Node failure recorded and handled under configured policy
T13	Log capture	Standard output/error written to expected paths
T14	GPU allocation (production only)	GPU requests respected and devices isolated

9. Queue Monitoring and Operational Commands

9.1 Essential commands

Command	Purpose
sinfo	Inspect partitions and nodes
squeue	Show queued and running jobs
sacct	View current and historical accounting records
scontrol show job JOBID	Detailed job information
scontrol show node c1	Detailed compute-node state
sprio -l	Inspect multifactor job priorities
sshare -l	Inspect historical fair-share usage
sstat -j JOBID	Inspect statistics for running job steps
scancel JOBID	Cancel an authorized job
sdiag	Scheduler diagnostic statistics

9.2 Monitoring examples

View queue:

```
squeue
```

View only pending jobs:

```
squeue -t PD
```

Monitor the test users:

```
squeue -u user1,user2
```

Identify why a job is pending:

```
scontrol show job <job_id>
```

View resource availability:

```
sinfo -o "%P %a %l %D %C %G"
```

View completed-job accounting:

```
sacct -S today \
  --format=JobID,User,Partition,State,Elapsed,ExitCode
```

View requested and observed resource usage:

```
sacct -j <job_id> \
  --format=JobID,State,Elapsed,AllocCPUS,ReqMem,MaxRSS,ExitCode
```

Some live usage fields depend on the job-accounting plugin and its collection configuration. Resource usage for individual job steps may differ from the parent allocation record.

9.3 Recommended monitoring metrics

Production monitoring should collect:

- Pending jobs by partition and reason.
- Queue wait-time median and 95th percentile.
- Running jobs by user and account.
- CPU, memory, and GPU allocation utilization.
- Job completion, failure, timeout, and OOM rates.
- Fair-share usage and priority distribution.
- Node availability and daemon health.
- SlurmDBD and database connectivity.
- GPU health, memory utilization, and thermal telemetry.

Prometheus and Grafana may be used for dashboards and alerting, with appropriately supported Slurm and NVIDIA DCGM exporters.

10. Failure Handling and Recovery

Failures should be classified by whether they originate from job code, resources, scheduling policy, or infrastructure.

Failure	Detection	Recovery approach
Application error	FAILED, nonzero exit code	Review stderr, fix code or environment, resubmit
Wall-time exceeded	TIMEOUT	Increase eligible time request or checkpoint workload
Memory exhausted	OUT_OF_MEMORY	Inspect memory usage, reduce batch size, request more RAM
Compute node unavailable	DOWN, DRAIN, NODE_FAIL	Diagnose node; restore or replace capacity
GPU error	NVIDIA health logs, job failure	Isolate unhealthy device/node and investigate
Controller restart	Controller service logs	Restore controller service and persistent state
Accounting unavailable	slurmdbd errors	Recover accounting service and database
Job log write failure	Missing logs or stderr errors	Restore storage availability and permissions
Job starvation	Prolonged PD	Examine priority, QOS, allocation limits, and fragmentation

10.1 Node recovery

Administrators can inspect affected nodes:

```
scontrol show node c1
```

Drain a node before maintenance:

```
scontrol update NodeName=c1 \
  State=DRAIN \
  Reason="Scheduled maintenance"
```

After correcting the underlying issue, an administrator may return the node to service:

```
scontrol update NodeName=c1 State=RESUME
```

Nodes should not be resumed until the underlying fault has been resolved.

10.2 Retry and requeue policy

Not all failures should be retried automatically. Infrastructure interruptions may warrant requeueing, whereas deterministic Python errors or exhausted memory requests generally require user intervention.

Jobs can be explicitly made requeueable:

```
#SBATCH --requeue
```

Authorized operators can request requeueing:

```
scontrol requeue <job_id>
```

A requeued job must be capable of restarting safely. For AI training, the application should write checkpoints and restore optimizer state, model weights, and training progress as appropriate.

Slurm requeueing does not automatically restore application state.

10.3 Fault-tolerance controls

Recommended production measures include:

1. Persistent Slurm controller state and configuration backups.
2. Durable database storage with scheduled backups.
3. Health checks for slurmctld, slurmd, and slurmdbd.
4. Checkpoint storage outside ephemeral job directories.
5. Graceful termination handling in long-running training jobs.
6. Clear administrative procedures for node draining and recovery.

A single-node GPU deployment cannot provide compute-node high availability. If the GPU node fails, its workloads cannot continue until capacity is restored or an additional eligible compute node is introduced.

11. Logging, Auditability, and Retention

11.1 Log categories

Log type	Source	Purpose
Job stdout/stderr	sbatch output/error directives	Application diagnostics
Controller logs	slurmctld	Scheduling decisions and infrastructure errors
Node daemon logs	slurmd	Execution and node-level diagnostics
Accounting data	SlurmDBD database	Historical workload usage and audit
GPU health logs	NVIDIA tooling	Hardware and device diagnostics
Authentication logs	OS/Munge	Access and authentication troubleshooting

11.2 Proposed retention policy

Data category	Proposed retention	Storage
Debug job logs	14 days	Persistent shared log volume
Training job logs	90 days	Persistent shared log volume
Slurm service logs	30 days online	Centralized log system
Job accounting metadata	1 year minimum	Backed-up accounting database
Training checkpoints	Project-specific	Durable checkpoint storage
Configuration backups	90 days or versioned	Protected backup storage

These are proposed operational defaults, not Slurm-enforced retention periods. They should be adjusted for storage capacity, institutional policy, and compliance obligations.

11.3 Log rotation and cleanup

For production service logs, use an OS-supported log rotation mechanism such as logrotate, configured with the deployment's logging behavior.

Job-log cleanup should use a scheduled retention process that:

- Deletes only files beyond the applicable retention period.
- Avoids deleting logs belonging to active jobs.
- Respects file ownership and project retention requirements.
- Records deletion or archival actions where required.

Do not run indiscriminate cleanup commands against a shared job directory.

12. Security and Isolation Controls

Production infrastructure should implement defense-in-depth controls.

Authentication: Use Munge for authenticated Slurm communication. Protect its key and limit controller administrative privileges.

Authorization: Enforce Unix user identities, Slurm associations, QOS restrictions, and administrative roles. Validate that normal users cannot change node states or manage other users' jobs.

Resource isolation: Use the supported Slurm cgroup plugins to constrain CPU cores, RAM, and GPU device access. Validate isolation with real simultaneous workloads.

Storage security: Use per-user or per-project directories with appropriate Unix permissions and shared-group policies. Job ownership does not automatically secure file contents.

Network security: Restrict Slurm service access to trusted infrastructure networks. Prefer controlled SSH access, and do not expose internal scheduler or database ports directly to the public internet.

Auditability: Preserve job ownership, timestamps, resource allocations, exit conditions, and administrative operations according to the retention policy.

Container-based user separation must not be presented as equivalent to production-grade GPU or kernel isolation without corresponding validation.

13. Deployment and Configuration Change Management

The deployment procedure should separate initial installation from subsequent configuration changes.

13.1 Initial installation

1. Install compatible Slurm components across controller and compute hosts.
2. Configure Munge authentication.
3. Configure and start the accounting database and SlurmDBD.
4. Register compute nodes and available resources.
5. Apply the partition and priority design.
6. Configure accounting users and QOS policies.
7. Configure cgroups and GPU GRES, where applicable.
8. Verify nodes, partitions, accounting, and job submission.
9. Execute the acceptance test matrix.

13.2 Configuration changes

Before applying a change:

- Back up the active configuration.
- Confirm the target Slurm version supports the proposed directives.
- Validate node and partition definitions.
- Assess whether a reconfiguration or daemon restart is required.
- Prefer a maintenance window for disruptive changes.

The standard administrative reconfiguration command is:

```
scontrol reconfigure
```

Not every configuration change can be applied without restarting Slurm services. Changes to resource-discovery configuration, plugins, and certain daemon settings require particular care.

13.3 Rollback

A rollback should restore the last verified configuration, recover necessary services, and rerun the basic node-registration, job-submission, and accounting tests.

Production changes should be version-controlled and documented with the reason, approver, change window, and validation results.

14. Docker-to-H100 Migration Plan

Area	Docker validation	H100 production validation
Nodes	c1, c2	gpu01
Scheduling	CPU/memory	CPU/memory/GPU
Multi-user jobs	Required	Required
Pending behavior	Required	Required
Fair-share	Accounting tests	Historical usage tests
Resource isolation	Container-dependent	Linux cgroups and GPU device constraints
Training	Simulated workloads	Real CUDA training
Monitoring	CLI commands	CLI, telemetry, alerting
Persistence	Docker volumes	Durable production storage
Recovery	Node/container restart	Hardware, daemon, and workload recovery

14.1 Production readiness gates

Team 7 should not declare the GPU deployment complete until the following are demonstrated:

- GPU enumeration identifies all eight expected H100 devices.
- Slurm reports the configured GPU GRES resources.
- A one-GPU job executes successfully.
- Two users can run separate GPU workloads without unintended device access.
- A multi-GPU workload can access exactly its allocation.
- An eight-GPU workload can run when the node is otherwise available.
- Insufficient GPU resources produce appropriate pending behavior.
- Accounting records GPU allocation and job outcomes.
- Runtime limits, resource constraints, and permitted cancellation operate correctly.
- Recovery procedures have been exercised and documented.

15. Risks, Limitations, and Open Decisions

Risk or decision	Impact	Proposed mitigation
One physical GPU node	Single point of failure	Checkpointing, backups, future node expansion
Shared debug/train partitions	Debug jobs can wait behind training workloads	Backfill, fair-share, evaluate reservations
Long training jobs	Extended resource occupation	Wall-time limits and per-user job caps
Incorrect resource declarations	Oversubscription or poor utilization	Validate node topology and enforce cgroups
Accounting database failure	Reduced visibility and accounting reliability	Persistent storage, backup, monitoring
GPU scheduling differences from Docker	Production-only bugs	Dedicated GPU acceptance testing
Excessive log growth	Storage exhaustion	Retention and monitored cleanup
Incomplete security isolation	Cross-user interference	Permissions, authentication, cgroups, device testing

The team must resolve the actual production node CPU/RAM specification, GPU topology, QOS enforcement strategy, storage design, and logging destination before the final production configuration is approved.

16. Project Deliverables and Acceptance Criteria

The following deliverables define completion of the Team 7 infrastructure queue design task.

Implementation verification checklist

0/11 verified

- [ ] Both Docker compute nodes are registered and healthy
- [ ] Debug and train partition configurations are verified
- [ ] user1 and user2 can submit independent jobs
- [ ] Concurrent execution is demonstrated
- [ ] Forced resource contention produces a Pending job
- [ ] A pending job becomes eligible after resources are released
- [ ] Unauthorized cross-user cancellation is rejected
- [ ] Job output and error logs are captured
- [ ] Completed and failed jobs appear in accounting
- [ ] Scheduling and failure test evidence is documented
- [ ] H100 migration requirements and limitations are recorded

16.1 Evidence requirements

For each completed acceptance test, retain the exact command used, submission user, Slurm job ID, execution timestamp, relevant squeue or sacct output, and observed result.

Screenshots or sanitized terminal captures should be included in the project submission, particularly for the two-user contention test.

16.2 Recommended documentation artifacts

The implementation repository should ultimately contain:

```
team7-infrastructure/
|-- README.md
|-- docs/
|   |-- queue-design.md
|   |-- architecture.md
|   |-- testing-results.md
|   `-- operations-runbook.md
|-- config/
|   |-- slurm.conf.example
|   |-- gres.conf.example
|   `-- cgroup.conf.example
|-- jobs/
|   |-- debug_job.sh
|   |-- train_simulation.sh
|   |-- exclusive_test.sh
|   `-- train_gpu.sh
`-- evidence/
    |-- node-status/
    |-- multi-user-test/
    |-- pending-state/
    `-- accounting/
```

17. Conclusion

Team 7's proposed architecture provides a scalable foundation for a multi-user Slurm workload management system.

The Docker implementation establishes the scheduling fundamentals: user identities, resource allocation, partition policies, job queuing, concurrent execution, accounting, and failure handling.

The future 8-H100 deployment extends these principles to accelerator workloads through GPU GRES, enforceable resource constraints, fair-share scheduling, monitoring, and persistent storage.

Final acceptance depends on observed test evidence rather than configuration alone. In particular, functional two-user scheduling in Docker must be demonstrated separately from production GPU isolation, performance, and reliability.

The recommended implementation order is to complete and document all Docker acceptance tests, validate accounting and fair-share behavior, and then conduct dedicated GPU scheduling and isolation tests on the target server.

Technical note: These configuration examples are design baselines, not a verified deployment. Slurm versions, installed plugins, container settings, hardware discovery, and existing configuration may require adjustments. Official reference: SchedMD Slurm Documentation